Software Requirements Specification

for

Automated Data Visualization Dashboard System

Version 0.1 - Initial Agile Draft

Course: Software Engineering and Modelling (SEM)

MIT WPU

Note: This is an early SRS prepared for Agile development.
Requirements may be updated after each sprint and feedback session.

Table of Contents

Revision History 3
1. Introduction 4
  1.1 Purpose 4
  1.2 Document Conventions 4
  1.3 Intended Audience 4
  1.4 Product Scope 4
  1.5 References 4
2. Overall Description 4
  2.1 Product Perspective 4
  2.2 Product Functions 4
  2.3 User Classes and Characteristics 5
  2.4 Operating Environment 5
  2.5 Design and Implementation Constraints 5
  2.6 User Documentation 5
  2.7 Assumptions and Dependencies 5
3. External Interface Requirements 6
  3.1 User Interfaces 6
  3.2 Hardware Interfaces 6
  3.3 Software Interfaces 6
  3.4 Communication Interfaces 6
4. System Features 6
  4.1 File Upload and Validation 6
  4.2 Data Processing and Summary 7
  4.3 Chart Selection and Generation 8
  4.4 Dashboard Display 8
  4.5 Export Results 9
  4.6 Optional AI Recommendation 9
  4.7 Error Handling 10
5. Other Nonfunctional Requirements 10
  5.1 Performance Requirements 10
  5.2 Safety Requirements 10
  5.3 Security Requirements 10
  5.4 Software Quality Attributes 10
  5.5 Business Rules 11
6. Other Requirements 11
Appendix A: Glossary 12
Appendix B: Analysis Model 12
Appendix C: To Be Determined List 12

The page numbers above are approximate and may change when the document is edited.

1. Introduction

1.1 Purpose

This document lists the initial software requirements for the Automated Data Visualization Dashboard System. The system will help a user upload a large spreadsheet and view useful summaries, charts, and key values without manually creating every graph.

This is not the final SRS. The Agile model has been selected, so the requirements may be improved, added, or removed after each sprint review and user feedback session.

1.4 Product Scope

Many users have large Excel files but may not know which chart or calculation is suitable for the data. The proposed system will reduce this manual work. It will read the file, identify common data types, calculate useful values, create more than one suitable chart, and show the results on a simple dashboard.

The first version will use normal programming rules for selecting graphs. An AI-based graph recommendation feature may be added later, but the basic system must work without AI.

1.5 References

- Provided “Software Requirements Specification” sample format.
- Project problem statement: automated visualization of large spreadsheet data.
- Agile process model write-up prepared for this SEM project.

2. Overall Description

2.1 Product Perspective

The product is a small, self-contained web application. The user opens it in a web browser, uploads a spreadsheet, and receives a dashboard. The first version may run on a local computer. A future version may be hosted online.

The current Jupyter and Python work may be used as an early prototype, but the final project structure can be improved during Agile sprints.

2.2 Product Functions

- Upload Excel or CSV files.
- Check whether the file and its columns are usable.
- Read and process the data.
- Find numeric, text, category, and date columns.
- Calculate values such as count, total, average, minimum, and maximum.
- Choose suitable graph types by using simple rules.
- Create several charts instead of showing only one graph.
- Display metrics and charts on an HTML dashboard.
- Show clear messages when the file contains an error.
- Optionally export selected results in a later sprint.

2.3 User Classes and Characteristics

User Type	Description	Main Needs
General User	A student, teacher, analyst, or office user with basic computer knowledge.	Upload a file and understand the results easily.
Developer / Admin	The person developing and maintaining the system.	Test the system, fix errors, and update chart rules.
Project Guide / Tester	The person checking whether the project meets its requirements.	Review features, test outputs, and give feedback.

2.4 Operating Environment

- A standard Windows or Linux computer.
- A modern browser such as Chrome, Edge, or Firefox.
- Python 3.x and required Python libraries.
- The first version may run locally without an internet connection.

2.5 Design and Implementation Constraints

- The project should use simple and mostly free tools.
- The core system must not depend on AI.
- The user interface should remain easy to understand.
- Very large or badly formatted files may take longer to process.
- The exact web framework may be selected during implementation.

2.6 User Documentation

- A one-page quick-start guide.
- Simple instructions on the upload screen.
- Clear error messages and basic help notes.

2.7 Assumptions and Dependencies

- The uploaded file has column headings in the first row.
- The file is not password-protected or badly damaged.
- The computer has enough memory to process the selected file.
- Required Python libraries are installed correctly.
- Users understand the basic meaning of charts and summary values.

3. External Interface Requirements

3.1 User Interfaces

The system will use a simple browser-based interface. The first screen will contain the project name, a file upload box, supported file types, and a Process Data button.

After processing, the dashboard should show summary cards, charts, column information, and any warnings. Buttons and messages should use clear words. The system should not show long technical error messages to normal users.

- Upload area for Excel or CSV files.
- Progress or loading message while data is being processed.
- Summary cards for rows, columns, missing values, and key calculations.
- Chart area containing multiple suitable graphs.
- Simple controls for choosing a column or chart, if added in a later sprint.

3.2 Hardware Interfaces

No special hardware is required. The system will use a normal computer, keyboard, mouse or touchpad, display, and local storage. Better memory and processor speed will help with large files.

3.3 Software Interfaces

- Python will be used for data processing.
- Pandas and OpenPyXL may be used to read and analyse spreadsheet files.
- A chart library such as Matplotlib or Plotly may be used to create graphs.
- HTML, CSS, and JavaScript will be used for the dashboard.
- The exact Python web framework is TBD and may be Flask or FastAPI.

3.4 Communication Interfaces

The browser and the Python application will communicate through HTTP when the application runs. The basic local version does not need email, cloud storage, or an outside internet service. If the system is hosted online later, HTTPS should be used.

4. System Features

The following features form the initial product backlog. Their details may change after sprint reviews.

4.1 File Upload and Validation

4.1.1 Description and Priority

The user uploads a spreadsheet. The system checks the file before processing it. Priority: High.

4.1.2 User Action and System Response

1. The user selects an Excel or CSV file.
2. The user clicks the process button.
3. The system checks the file type, file size, and basic structure.
4. The system accepts the file or shows a clear error message.

4.1.3 Functional Requirements

ID	Requirement	Priority	Status

ID	Requirement	Priority	Status
FR-01	The system shall allow the user to select an .xlsx, .xls, or .csv file.	High	Planned
FR-02	The system shall reject unsupported file types.	High	Planned
FR-03	The system shall check that the file contains at least one data row and one column.	High	Planned
FR-04	The system shall show a simple message when the file cannot be read.	High	Planned
FR-05	The maximum file size for the first version is TBD.	Medium	TBD

4.2 Data Processing and Summary

4.2.1 Description and Priority

The system reads the data, identifies common column types, and calculates useful summary values. Priority: High.

4.2.2 Functional Requirements

ID	Requirement	Priority	Status
FR-06	The system shall count the number of rows and columns.	High	Planned
FR-07	The system shall identify numeric, text, category, and date columns where possible.	High	Planned
FR-08	The system shall count missing or empty values.	High	Planned
FR-09	For numeric columns, the system shall calculate count, average, minimum, and maximum.	High	Planned
FR-10	The system should also calculate totals when a total is meaningful.	Medium	Planned
FR-11	The system shall not change the original uploaded file.	High	Planned

4.3 Chart Selection and Generation

4.3.1 Description and Priority

The system chooses suitable charts by following fixed rules. It should create a group of charts that explains the whole dataset rather than outputting only one graph. Priority: High.

4.3.2 Basic Chart Rules

- Use a line chart when values change over time or in a clear order.
- Use a bar chart to compare categories.
- Use a pie chart only when there are a small number of categories that form parts of a whole.
- Use a histogram to show the spread of a numeric column.
- Use a scatter plot when comparing two numeric columns may show a relationship.

4.3.3 Functional Requirements

ID	Requirement	Priority	Status
FR-12	The system shall select chart types using programmed rules.	High	Planned
FR-13	The system shall generate more than one chart when the data supports it.	High	Planned
FR-14	The system shall avoid a pie chart when there are too many categories.	High	Planned
FR-15	Every chart shall contain a clear title and labels.	High	Planned
FR-16	The system should show important high points, low points, or changes when useful.	Medium	Planned
FR-17	The system shall skip a chart when the selected data is not suitable for it.	High	Planned

4.4 Dashboard Display

4.4.1 Description and Priority

The results are shown together on one dashboard so the user can understand the file quickly. Priority: High.

4.4.2 Functional Requirements

ID	Requirement	Priority	Status
FR-18	The dashboard shall show the uploaded file name.	High	Planned
FR-19	The dashboard shall show key summary cards.	High	Planned
FR-20	The dashboard shall show multiple charts in an organised layout.	High	Planned
FR-21	The dashboard shall remain readable on a normal laptop screen.	High	Planned
FR-22	The dashboard should allow the user to select a column or graph in a later sprint.	Medium	Backlog
FR-23	The dashboard should show warnings for missing or unusual data.	Medium	Planned

4.5 Export Results

4.5.1 Description and Priority

The user may need to save charts or the dashboard. This feature is useful but is not necessary for the first working version. Priority: Medium.

4.5.2 Functional Requirements

ID	Requirement	Priority	Status
FR-24	The system should allow a chart to be saved as an image.	Medium	Backlog
FR-25	The system may allow the dashboard to be exported as PDF or HTML.	Low	TBD
FR-26	The exact export types shall be decided after user feedback.	Low	TBD

4.6 Optional AI Recommendation

4.6.1 Description and Priority

An AI feature may later recommend a chart or explain why a chart was selected. The project is not dependent on this feature. Priority: Low / Future.

4.6.2 Functional Requirements

ID	Requirement	Priority	Status
FR-27	The core dashboard shall work even when the AI feature is not available.	High	Planned
FR-28	The AI feature may suggest additional chart types in a future sprint.	Low	Future
FR-29	The user should be able to ignore an AI suggestion.	Low	Future

4.7 Error Handling

4.7.1 Description and Priority

The system should handle common problems without crashing. Priority: High.

4.7.2 Functional Requirements

ID	Requirement	Priority	Status
FR-30	The system shall show a useful message for an empty, damaged, or unsupported file.	High	Planned
FR-31	The system shall handle missing values without stopping the complete process.	High	Planned
FR-32	The system shall record technical errors for the developer when possible.	Medium	Backlog
FR-33	The system shall allow the user to try another file after an error.	High	Planned

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The first version should process a normal classroom test file within 30 seconds on a standard laptop.
- After processing is complete, the dashboard should open within 5 seconds.
- Exact limits for very large files will be tested and updated in a later sprint.
- The system should display a loading message when processing takes time.

5.2 Safety Requirements

This software has no physical safety risk. The main protection is that the original uploaded file must not be edited or deleted. The system should work on a copy of the data in memory or temporary storage.

5.3 Security Requirements

- The first local version should not send uploaded data to an outside service.
- Temporary uploaded data should be removed when it is no longer needed.
- File names and input values should be checked before use.
- If the system is hosted online later, user access and HTTPS will be required.

5.4 Software Quality Attributes

Quality	Initial Requirement
Ease of use	A new user should be able to upload a file and view results without training.
Correctness	Summary values and charts should match the uploaded data.
Reliability	A problem in one column should not stop all useful results when possible.
Maintainability	Chart rules and processing steps should be divided into clear modules.
Compatibility	The dashboard should work on current versions of common browsers.
Testability	Important functions should be testable with small sample files.

5.5 Business Rules

- Only supported spreadsheet formats may be processed.
- The original uploaded file must remain unchanged.
- A pie chart should not be used for a large number of categories.

- The system should prefer easy-to-read charts over decorative charts.
- The optional AI feature must not block the normal rule-based system.

6. Other Requirements

6.1 Agile Development Requirements

The project will be developed in short Agile sprints. At the start of each sprint, a small group of requirements will be selected from the product backlog. At the end of the sprint, the working result will be shown to the teacher, guide, or sample users. Their feedback may change the next version of this SRS.

- Sprint 1: Basic file upload, validation, and simple data reading.
- Sprint 2: Summary values and basic graph rules.
- Sprint 3: Dashboard design and multiple charts.
- Sprint 4: Error handling, testing, and improvements.
- Later sprint: Export, filters, or optional AI recommendation, depending on time.

6.2 Testing Requirements

- Use small, medium, and large sample files for testing.
- Test files with missing values, wrong dates, text, and numeric columns.
- Check calculated values against manual calculations.
- Check that selected graph types follow the stated rules.
- Ask at least one sample user to test the dashboard and give feedback.

6.3 Data Storage Requirements

A database is not required for the first version. The uploaded file may be processed temporarily. A database may be considered later only if users need saved projects, login accounts, or past reports.

Appendix A: Glossary

Term	Simple Meaning
Agile	A way of developing software in small steps and improving it using regular feedback.
Dashboard	A screen that shows important values and charts together.
Dataset	A collection of data, usually arranged in rows and columns.
Metric	A useful measured value such as total, average, minimum, or maximum.
CSV	A simple text file that stores table data separated by commas.
Validation	Checking whether an input file is acceptable before using it.
Functional Requirement	A task or action the system must perform.
Nonfunctional Requirement	A quality requirement such as speed, ease of use, or security.
Product Backlog	A changing list of features and improvements planned for future sprints.
TBD	To Be Determined; a detail that will be decided later.

Appendix B: Analysis Model

The following diagram shows the basic flow of the system and how Agile feedback is used to update the next sprint.

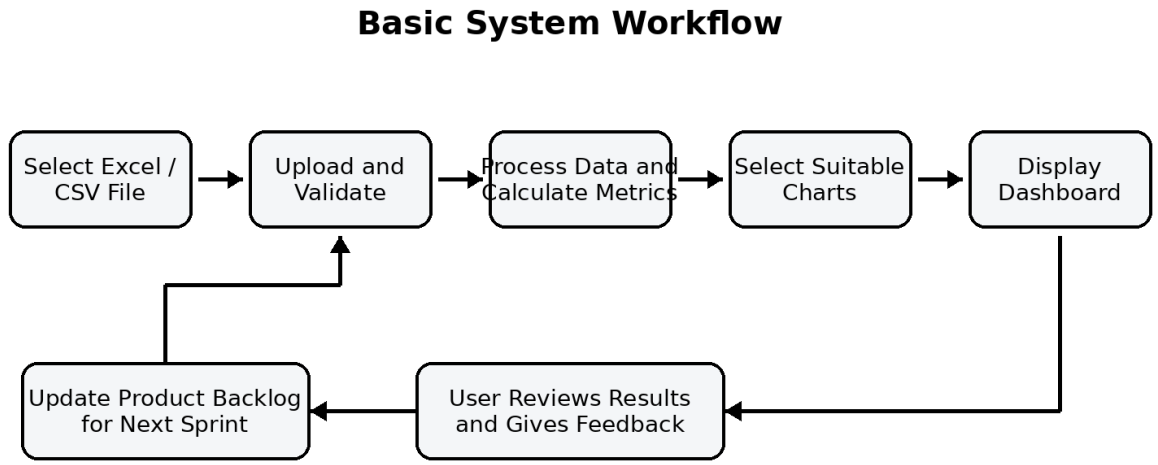


Figure B-1: Basic system workflow and Agile feedback loop

Appendix C: To Be Determined List

1. Maximum supported file size for the first version.
2. Final Python web framework: Flask, FastAPI, or another simple option.
3. Exact chart library used in the dashboard.
4. Whether filters and column selection will be included in the first release.
5. Export formats: image, PDF, HTML, or a combination.
6. Whether the optional AI recommendation feature will be implemented.
7. Final performance limits after testing with large files.